

Proving the hard-to-round search

Code: `code/APaul/htr.c` and `code/APaul/htrplan/`. Numbers are marked **read** (off the code), **measured** (a run) or **computed**.

1 Definitions

- e is the exponent of $\exp x$: $2^{e-1} \leq \exp x < 2^e$.
- $Y(x) = \exp(x) \cdot 2^{54-e}$, in $[2^{53}, 2^{54})$. The integers of this scale are the doubles and the midpoints between them: the **grid**.
- x is **hard to round at level m** when $\text{dist}(Y(x), \mathbb{Z}) < 2^{-m}$: the m bits after the round bit of $\exp x$ are all equal.
- u is the ulp of x : the distance from x to the next double.

2 The theorem

For an interval $[x_0, x_1)$ and a level m :

$$(S) \quad \forall x \in [x_0, x_1), x \text{ hard to round at level } m \Rightarrow (x, v) \in a \text{ with } v \neq 0.$$

search fills the caller's array a with the candidates (x, v) , where the **verdict** v is 1 (hard), 0 (not hard) or 2 (undecided), and returns their number; (S) holds when that number fits in a (otherwise the caller calls again with a larger a). The array may hold false cases, never miss a true one.

3 How the search works

The search cuts $[x_0, x_1)$ into **chunks** of $n = 2^{20}$ consecutive doubles $x + iu$, $0 \leq i < n$. On a chunk, Y is close to a **line**, since the derivative of $\exp$ is $\exp$:

$$Y(x + iu) \approx Y(x) + i \cdot Y(x)u.$$

Only the fractional parts matter. They are kept as 64-bit integers (the unit is 2^{-64}):

- $A \approx$ the fractional part of $Y(x)$, $B \approx$ that of the slope $Y(x)u$;
- the **drift** is how far Y moves away from the line over the chunk, at most $Y(x)u^2n^2/2$;
- the **window** is $E = 2^{64-m} + \text{drift}$.

The inner loop adds B to A n times. Each i where A is within E of an integer is a **candidate**: it is checked with a precise evaluation and added to the array with its verdict.

Why it works. If $x + iu$ is hard, Y is within 2^{-m} of an integer, so the line is within $2^{-m} + \text{drift} + \text{errors of it}$. If the window covers that, i is a candidate.

4 The three programs

- **Program 1**, `htr.c`: evaluates $\exp$ with MPFR in three functions. Not proved; it is the reference (Section A).
- **Program 2**, the generic search: `htr.c` with three **evaluators** of $\exp$ as parameters, `eval_seed`, `eval_low`, `eval_check`. An evaluator only promises a bound on $\exp x$. They are its only external functions: the exponent e and the check of a candidate are code of Program 2, built on them, and the result is an array, not a printout. Proved once, for all evaluators that meet their specifications.
- **Program 3**: evaluators in integer arithmetic, proved to meet the specifications.

Programs 2 and 3 are **self-contained**: they call no library, not even `libc` or `libm`. The integers are arrays of 32-bit limbs with our own operations, and doubles are read and built from their 64 bits (`dbl.h`), instead of `libm`'s `frexp`, `ldexp` and `nextafter`. Only the driver, `main.c`, uses `libc`, to read the arguments and print the array; it is outside the proof.

5 The specifications and the dependencies

The evaluators.

- `eval_seed(x)` gives the line at each chunk start, and `eval_check(x)` the value to decide a candidate. Both return three doubles $h + l + s \approx \exp x$.
- `eval_low(x)` returns one double d , a lower bound of $\exp x$.

	specification	of
E1	$ h + l + s - \exp x \leq 2^{-\kappa} \exp x$	<code>eval_seed</code> , <code>eval_check</code>
E2	$ l < 2^{e-53}$ and $ s < 2^{e-54}$	<code>eval_seed</code> , <code>eval_check</code>
E3	$d \leq \exp x < d + \text{ulp}(d)$	<code>eval_low</code>

What each evaluator is for. The three answer different questions, are called at different rates, and need different precisions (κ is the number of correct bits):

evaluator	question	calls	precision needed
<code>eval_low</code>	which binade is $\exp x$ in?	2 per interval	a few bits, but from below
<code>eval_seed</code>	where does the line start, with what slope?	1 per chunk	its error must fit in the margin of the window (2c)
<code>eval_check</code>	is this candidate hard?	1 per candidate, about 41 per chunk (read)	about $54 + m$ bits and a margin: about 95 for $m = 35$ (computed)

So the evaluator to make fast is `eval_check`, and it is not the one that needs the most precision. A lower precision only widens the undecided band, and undecided candidates are kept. The $\kappa \geq 118$ that `search.h` asks of `eval_check` is a choice, not a requirement: it keeps the error below one unit of 2^{-64} , which makes 2d simple. Program 3 gives 158 for all three.

The dependencies. Each line needs the lines below it.

```
(S) every hard x in [x0, x1) is in the array, with a verdict v != 0
|-- 2a the exponent e is exact           needs E3           to do, easy
|-- 2b the conversions to 64 bits         needs E2, 2a       to do
|-- 2c the window covers the errors       needs E1           to do, HARD
|-- 2d the check never rejects a hard x   needs E1 E2        to do, easy
|-- 2e the inner loop                    done, Scan.v
`-- 2f the chunks cover [x0, x1)         done, ScanAll.v

E1..E3 for Program 3
|-- (E) the integer y is within 2^-160 of exp x           to do
|-- 3a (E) + the split of y                             => E1 with kappa = 158, E2
`-- 3b (E) + y - err cut to 53 bits                       => E3
```

6 Program 2: the generic search

Interface, `search.h`:

```
// Program 2, the generic search: htr.c with the evaluators as parameters.
//
// x is hard to round at level m when dist (exp(x) 2^(54-e), Z) < 2^-m,
// where e is the exponent of exp(x): 2^(e-1) <= exp(x) < 2^e.
// The theorem:
// (S) for every double x in [x0, x1) hard to round at level m,
```

```

//      if search returns nc <= cap, a[0..nc) holds (x, v) with v != 0,
// provided the three evaluators meet their specifications below. They
// are the only external functions: the exponent and the check are built
// on top of them in search.c.

#include <stdint.h>

// eval_seed (&h, &l, &s, x) gives the line at each chunk start, as three
// doubles h + l + s, with
// (E1) |h + l + s - exp(x)| <= 2^-kappa * exp(x)
// (E2) |l| < 2^(e-53) and |s| < 2^(e-54)
// kappa is fixed by the proof of the window: about 118 is enough
// (computed, not proved). E2 lets the search read the bits after the
// round bit from l and s alone.
typedef void (*seed_t) (double *h, double *l, double *s, double x);

// eval_low (x) gives exp(x) from below, as one double d, with
// (E3) d <= exp(x) < d + ulp(d)
// so that d and exp(x) have the same exponent.
typedef double (*low_t) (double x);

// eval_check (&h, &l, &s, x) decides each candidate. It must meet E1 and
// E2 with kappa >= KAPPA_CHECK. Its error is then at most one unit of
// 2^-64 on the fractional part of exp(x) 2^(54-e), and the conversion to
// 64 bits adds two more: CHECK_ERR. KAPPA_CHECK is a choice that keeps
// the proof simple: about 95 bits would do for m = 35, with a larger
// CHECK_ERR.
typedef void (*check_t) (double *h, double *l, double *s, double x);
#define KAPPA_CHECK 118
#define CHECK_ERR 3

// A candidate x with the verdict v of the check: 1 hard, 0 not hard,
// 2 undecided.
struct cand { double x; int v; };

// search stores the candidates, in increasing order, in the caller's array
// a of capacity cap, and returns their number nc. When nc > cap, only the
// first cap are stored, and the caller must call again with a larger a.
unsigned long search (seed_t eval_seed, low_t eval_low, check_t eval_check,
                     double x0, double x1, int m,
                     struct cand *a, unsigned long cap);

```

Code: htr.c's search, with the calls to MPFR replaced by the evaluators, a new function check on top of eval_check, the calls to libm replaced by dbl.h (frexp by dexpo, ldexp by dscale, nextafter by dpred), and the candidates stored in the caller's array. The complete code, search.c:

```

// Program 2, the generic search: the search of htr.c, line for line,
// with dd_exp replaced by eval_seed, ref_exp by eval_low, and check by
// a check written here on top of eval_check. Self-contained: libm's
// frexp, ldexp and nextafter are replaced by dexpo, dscale and dpred
// (dbl.h), and the array is the caller's.

#include "dbl.h"
#include "search.h"

```

```

// given x, -1 < x < 1, return the 64-bit value corresponding
// to the fractional part of x
static uint64_t
get_uint64 (double x)
{
    assert (-1 < x && x < 1);
    x = dscale (x, 64);
    if (x >= 0) return x;
    uint64_t ret = (uint64_t) (-x);
    return ~ret + (uint64_t) 1;
}

// is x hard to round at level m: 1 yes, 0 no, 2 undecided. e is the
// exponent of exp(x). F is the fractional part of exp(x) 2^(54-e), times
// 2^64, read from l and s as A is; d is its distance to the nearest
// integer; F is within CHECK_ERR of the exact value (E1, E2).
static int
check (check_t eval_check, int e, double x, int m)
{
    double h, l, s;
    eval_check (&h, &l, &s, x);
    l = dscale (l, 54 - e);
    if (l >= 1.0) l -= 1.0;
    while (l < 0) l += 1.0;
    s = dscale (s, 54 - e);
    uint64_t F = get_uint64 (l) + get_uint64 (s);
    uint64_t d = F < -F ? F : -F;
    uint64_t T = 1ul << (64 - m);
    if (d + CHECK_ERR < T) return 1;
    if (d >= T + CHECK_ERR) return 0;
    return 2;
}

// add the candidate x with its verdict v to the array a of capacity cap,
// if there is room; count it in any case
static void
report (struct cand *a, unsigned long cap, unsigned long *nc, double x,
        int v)
{
    if (*nc < cap) {
        a[*nc].x = x;
        a[*nc].v = v;
    }
    (*nc) ++;
}

// search hard-to-round cases of exp in [x0,x1)
// with at least m identical bits after round bit
unsigned long
search (seed_t eval_seed, low_t eval_low, check_t eval_check,
        double x0, double x1, int m, struct cand *a, unsigned long cap)
{
    unsigned long nc = 0;
    uint64_t n = 1ul << 20;
    int e, e0, e1;

```

```

// check ulp(x0) = ulp(nextbelow(x1))
e0 = dexpo (x0);
e1 = dexpo (dpred (x1));
assert (e0 == e1);

double ux = pow2 (e0 - 53); // ux = ulp(x)

// check exp(x) lies in the same binade in [x0, x1)
e = dexpo (eval_low (x0));
e1 = dexpo (eval_low (dpred (x1)));
assert (e == e1);

double h, l, s;
double x = x0;
while (x < x1) {
    eval_seed (&h, &l, &s, x);
    double dh = h * ux, dl = l * ux; // 1st derivative, multiplied by ux
    double dd = h * ux * ux; // 2nd derivative, multiplied by ux^2

    // prepare loop
    // A is the bits of h+l after the round bit
    h = dscale (h, 54 - e); // now ulp(h) = 2
    l = dscale (l, 54 - e);
    // we now have -2 < l < 2
    if (l >= 1.0) l -= 1.0;
    while (l < 0) l += 1.0;
    assert (0 <= l && l < 1);
    s = dscale (s, 54 - e);
    uint64_t lu = get_uint64 (l), su = get_uint64 (s);
    uint64_t A = lu + su;
    // scale the derivatives too
    dh = dscale (dh, 54 - e);
    dl = dscale (dl, 54 - e);
    dd = dscale (dd, 54 - e);
    uint64_t B = get_uint64 (dh) + get_uint64 (dl);
    // the maximal error is bounded by dd/2*n^2
    dd = dd / 2.0 * n * n;
    // hard-to-round cases are at distance < 2^{-(m+1)} ulp(h)
    uint64_t E = get_uint64 (dd) + (1ul << (64 - m));
    // we want to find values of A in [-E, E]
    A += E; // shift A
    // now we want to find values of A in [0, 2*E]
    for (uint64_t i = 0; i < n; i++) {
        if (__builtin_expect (A < 2*E, 0)) { // found potential hard-to-round case
            double xi = x + i * ux;
            report (a, cap, &nc, xi, check (eval_check, e, xi, m));
        }
        A += B;
    }
    x += n * ux;
    if (x > x1)
        n = (x1 - x) / ux;
}
return nc;
}

```

and `dbl.h`, the doubles read and built from their bits, shared with Program 3:

```
// Doubles (binary64), read and built from their 64 bits, with no library:
// what Programs 2 and 3 use instead of libm's frexp, ldexp and nextafter.
// A failed check stops the program with the compiler's trap.

#include <stdint.h>

#define assert(c) do { if (!(c)) __builtin_trap (); } while (0)

union dbits { double d; uint64_t u; };

// 2^E, for -1022 <= E <= 1023
static inline double
pow2 (int E)
{
    union dbits b;
    b.u = (uint64_t) (E + 1023) << 52;
    return b.d;
}

// v 2^E, as ldexp: exact while v 2^E is normal, rounded once otherwise
static inline double
dscale (double v, int E)
{
    if (E > 1023) { v *= pow2 (1023); E -= 1023; }
    if (E < -1022) { v *= pow2 (-1022); E += 1022; }
    return v * pow2 (E);
}

// the exponent e of v, 2^(e-1) <= |v| < 2^e, as frexp; v normal
static inline int
dexpo (double v)
{
    union dbits b;
    b.d = v;
    int be = (b.u >> 52) & 0x7ff;
    assert (be != 0 && be != 0x7ff);
    return be - 1022;
}

// the double just below v, as nextafter (v, -infinity); v finite
static inline double
dpred (double v)
{
    union dbits b;
    b.d = v;
    if (v > 0) b.u --;
    else if (v < 0) b.u ++;
    else b.u = 0x8000000000000001ul; // -0 and +0: -2^-1074
    return b.d;
}
```

Back to Program 1. `eval_seed = eval_check = htr.c's dd_exp ( $\kappa \approx 158$ )`, `eval_low = htr.c's ref_exp` (rounding toward zero meets E3). The candidates are then exactly `htr.c's`. The decision

may differ from `htr.c`'s `check` only within `CHECK_ERR` of the threshold, where Program 2 answers undecided; on the test slices the two agree (Section D). The exact instantiation is in Section B.

Obligations.

- **2a.** e is the exponent of `eval_low`(x), exact by E3, since $\exp x < d + \text{ulp}(d) \leq 2^e$.
- **2b.** h, l, s are scaled by 2^{54-e} and cut into 64-bit integers (`get_uint64`), with sums modulo 2^{64} . This reads the right bits only if e is exact (2a) and l, s are small (E2).
- **2c.** The code's window is $2^{64-m} + \text{drift}$ (`read`). Four errors are not in it: `eval_seed`'s (κ); the drift uses the curvature at the start of the chunk, not its maximum; the truncations of **B** add up over n steps; **A** is truncated. The proof must show they fit in the margin, or add a term to **E**.
- **2d.** `check` reads the fractional part F of $Y(x)$ as the search reads **A**, within `CHECK_ERR` = 3 units of 2^{-64} : one for E1, two for the truncations. With d the distance of F to the nearest integer, it answers 1 if $d + 3 < 2^{64-m}$, 0 if $d \geq 2^{64-m} + 3$, 2 otherwise. So 0 means $\text{dist}(Y(x), \mathbb{Z}) \geq 2^{-m}$.
- **2e, 2f.** Done: `Scan.v`, `ScanAll.v`.

7 Program 3: the proved evaluator

`eval_fix.c` provides `eval_fix`, used as `eval_seed` and `eval_check`, and `low_fix`, used as `eval_low`. Both start from `fix_exp`, which computes an integer y and an integer k with $\exp x \approx y \cdot 2^{k-200}$, by Tang's method:

1. write $x = N \ln 2/256 + r$, with N an integer and $|r| \leq 2^{-9.5}$; then $N = 256k + j$ with $0 \leq j < 256$;
2. compute $\exp r$ with its Taylor polynomial of degree 13;
3. multiply by $2^{j/256}$, taken from a table of 256 constants; then $\exp x = 2^k \cdot 2^{j/256} \cdot \exp r$.

Every quantity is an integer Z standing for $Z \cdot 2^{-200}$. An integer is 8 limbs of 32 bits (256 bits, two's complement for a sign), and the code has its own add, subtract, multiply-then-shift, divide by a small integer, compare and bit length, as `htr_plain.c` does. The proof of (E) is over $\mathbb{Z}$; one lemma per limb operation ties it to the limbs. Code: Section C.

The one bound:

$$(E) \quad |y \cdot 2^{k-200} - \exp x| \leq \text{err} \cdot 2^{k-200}, \quad \text{err} = 2^{40}.$$

Why (E) is all that matters. Program 3 never rounds in MPFR's sense: every operation is on integers, with an explicit floor. Each decision is taken on the interval $[y - \text{err}, y + \text{err}]$, which contains the exact value by (E): the exponent and `eval_low` stop on an `assert` when the interval does not decide, and the check answers undecided. So a lack of precision gives more undecided answers or a stop, never a wrong answer. Correctness rests on (E) alone. (E) is not proved yet; the evidence so far is the error budget below (**computed**), ten points against MPFR at 400 bits, and the test slices (**measured**, Section D).

From (E) to E1–E3.

- **3a.** h, l, s are the first three blocks of 53 bits of y : they lose less than 2^{-158} relative (E1, $\kappa = 158$), and each is below the last bit of the one before (E2). E2 is relative to e : `fix_exp` asserts that y is not within `err` of a power of 2, so y has the exponent of $\exp x$.
- **3b.** d is $y - \text{err}$ cut down to 53 bits, so $d \leq \exp x$. The code asserts that $y + \text{err}$ stays below the next 53-bit value, so $\exp x < d + \text{ulp}(d)$ (E3).

Proof of (E). One lemma per step of `fix_exp`. Errors in units of 2^{-200} , **computed**:

step	error	proof
r	2 units	$\ln 2/256$ certified by <code>interval</code>
$ r \leq 2^{-9.5}$	—	an <code>assert</code> in the code
Horner	about 4 units	one floor per step

Taylor remainder	2^{-169}	one interval call
$\times 2^{j/256}$	1.5 units	256 constants, each certified by interval
total	$\leq 2^{-160}$	8 bits to spare

8 What to do

1. 2a to 2d: proves (S) for any evaluators, and fixes κ .
2. (E), 3a and 3b: Program 3 meets the specifications.
3. The whole range: Program 2 only works for $|x| < 1/2$, one binade at a time, and above $x = -634$ (Section E).
4. Tie the Rocq models to the C (or its Capla port): not tried yet.

A Annex: Program 1, `htr.c`

MPFR is used in `dd_exp` (`eval_seed`, `eval_check`), `ref_exp` (`eval_low`) and `check`, which Program 2 replaces by its own check.

```
// search hard-to-round cases of exp in [0.25,0.5)
// with at least 43 identical bits after round bit

#include <stdio.h>
#include <stdint.h>
#include <mpfr.h>
#include <assert.h>
#include <math.h>

// reference code using MPFR, with RNDZ
double
ref_exp (double x)
{
    mpfr_t y;
    mpfr_exp_t emin = mpfr_get_emin ();
    mpfr_set_emin (-1073);
    mpfr_init2 (y, 53);
    mpfr_set_d (y, x, MPFR_RNDN);
    int inex = mpfr_exp (y, y, MPFR_RNDZ);
    mpfr_subnormalize (y, inex, MPFR_RNDZ);
    double ret = mpfr_get_d (y, MPFR_RNDN);
    mpfr_clear (y);
    mpfr_set_emin (emin);
    return ret;
}

// put in h+l+s a triple-double approximation of exp(x)
static void
dd_exp (double *h, double *l, double *s, double x)
{
    mpfr_t t;
    mpfr_init2 (t, 3*53+2);
    mpfr_set_d (t, x, MPFR_RNDN);
    mpfr_exp (t, t, MPFR_RNDN);
    *h = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *h, MPFR_RNDN);
    *l = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *l, MPFR_RNDN);
}
```

```

    *s = mpfr_get_d (t, MPFR_RNDN);
    mpfr_clear (t);
}

unsigned long checks = 0, found = 0;

static void
check (double x, int m)
{
    // printf ("check x=%la\n", x);
    checks ++;
    mpfr_t y, z;
    mpfr_init2 (y, 54);
    mpfr_init2 (z, 53 + m);
    mpfr_set_d (y, x, MPFR_RNDN);
    mpfr_exp (z, y, MPFR_RNDN);
    mpfr_set (y, z, MPFR_RNDN);
    if (mpfr_cmp (y, z) == 0) {
        printf ("%la\n", x);
        found ++;
    }
    mpfr_clear (y);
    mpfr_clear (z);
}

// given x, -1 < x < 1, return the 64-bit value corresponding
// to the fractional part of x
static uint64_t
get_uint64 (double x)
{
    assert (-1 < x && x < 1);
    x = ldexp (x, 64);
    if (x >= 0) return x;
    uint64_t ret = (uint64_t) (-x);
    return ~ret + (uint64_t) 1;
}

// search hard-to-round cases of exp in [x0,x1)
// with at least m0 identical bits after round bit
static void
search (double x0, double x1, int m)
{
    uint64_t n = 1ul << 20;
    int e, e0, e1;

    // check ulp(x0) = ulp(nextbelow(x1))
    frexp (x0, &e0);
    frexp (nextafter (x1, x0), &e1);
    assert (e0 == e1);

    double ux = ldexp (1.0, e0 - 53); // ux = ulp(x)

    // check exp(x) lies in the same binade in [x0, x1)
    double y0 = ref_exp (x0);
    double y1 = ref_exp (nextafter (x1, x0));
    frexp (y0, &e);

```

```

frexp (y1, &e1);
assert (e == e1);
double uh = ldexp (1.0, e - 53); // uh = ulp(exp(x))

double h, l, s;
double x = x0;
while (x < x1) {
    dd_exp (&h, &l, &s, x);
    double dh = h * ux, dl = l * ux; // 1st derivative, multiplied by ux
    double dd = h * ux * ux; // 2nd derivative, multiplied by ux^2

    // prepare loop
    // A is the bits of h+l after the round bit
    h = ldexp (h, 54 - e); // now ulp(h) = 2
    l = ldexp (l, 54 - e);
    // we now have -2 < l < 2
    if (l >= 1.0) l -= 1.0;
    while (l < 0) l += 1.0;
    assert (0 <= l && l < 1);
    s = ldexp (s, 54 - e);
    uint64_t lu = get_uint64 (l), su = get_uint64 (s);
    uint64_t A = lu + su;
    // scale the derivatives too
    dh = ldexp (dh, 54 - e);
    dl = ldexp (dl, 54 - e);
    dd = ldexp (dd, 54 - e);
    uint64_t B = get_uint64 (dh) + get_uint64 (dl);
    // the maximal error is bounded by dd/2*n^2
    dd = dd / 2.0 * n * n;
    // hard-to-round cases are at distance < 2^-(m+1) ulp(h)
    uint64_t E = get_uint64 (dd) + (1ul << (64 - m));
    // we want to find values of A in [-E, E]
    A += E; // shift A
    // now we want to find values of A in [0, 2*E]
    for (uint64_t i = 0; i < n; i++) {
        if (__builtin_expect (A < 2*E, 0)) { // found potential hard-to-round case
            double xi = x + i * ux;
            check (xi, m);
        }
        A += B;
    }
    x += n * ux;
    if (x > x1)
        n = (x1 - x) / ux;
}
}

/* With x0 = 0.25, x1 = 0.25001, m = 35
on thym.loria.fr (Intel(R) Core(TM) Ultra 7 265) with gcc 15.3.0
(-O3 -march=native):

0x1.00002385331bep-2
0x1.0000561fc7e06p-2
0x1.0001cd08ef3d4p-2
0x1.00026c769e211p-2
0x1.00028a78a7303p-2

```

```
7056503 checks, found 5 hard-to-round
```

```
real 0m39.080s
user 0m39.032s
sys 0m0.008s
```

BaCSeL:

```
make GMP=$GMP MPFR=$MPFR FPLLL=$FPLLL DEFSAL="-DEXP -DBASIS=2 -DAUTOMATIC"
zimmerma@thym:~/svn/bacsel$ ./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t
18 -t0 4503599627370496 -t1 4503779771355591 -d 2 -alpha 2 -e_in -2 -nthreads 4
zimmerma@thym:~/svn/bacsel$ ./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t
18 -t0 4503599627370496 -t1 4503779771355591 -d 2 -alpha 2 -e_in -1 -nthreads 4
./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t 18 -t0 4503599627370496 -t1
4503779771355591 -d 2 -alpha 2 -e_in -1 -nthreads 4
Welcome to BaCSeL version 4.0, using 4 thread(s)
Compilation flags: -DBASIS=2 -DEXP -DAUTOMATIC
LLL parameters: delta=0.999 eta=0.501
Parameters for AUTOMATIC: FAILURE_LOWERBOUND=2.0 FAILURE_UPPERBOUND=4.0
*** x=0x4.0001587f1f818p-4, distance is: 1.059463030e-11 0x1.0000561fc7e06p-2
*** x=0x4.00008e14cc6f8p-4, distance is: 1.055287224e-11 0x1.00002385331bep-2
*** x=0x4.00073423bcf5p-4, distance is: 2.577853443e-12 0x1.0001cd08ef3d4p-2
*** x=0x4.000a29e29cc0cp-4, distance is: 7.860279955e-12 0x1.00028a78a7303p-2
*** x=0x4.0009b1da78844p-4, distance is: 7.913797859e-12 0x1.00026c769e211p-2
*/
int main()
{
    double x0 = 0.25, x1 = 0.25001;
    int m = 35;
    search (x0, x1, m);
    printf ("%lu checks, found %lu hard-to-round\n", checks, found);
}
```

B Annex: the MPFR instantiation

Program 1 is recovered by the call

```
search (eval_mpfr, low_mpfr, eval_mpfr, x0, x1, m)
```

(main.c), with the two functions of eval_mpfr.c. eval_mpfr is htr.c's dd_exp unchanged; low_mpfr is htr.c's ref_exp unchanged.

```
// The evaluators of the initial program (htr.c): exp with MPFR, trusted.

#include <stdio.h>
#include <mpfr.h>
#include "eval.h"

// htr.c's ref_exp: exp(x) rounded toward zero, which meets E3
double
low_mpfr (double x)
{
    mpfr_t y;
    mpfr_exp_t emin = mpfr_get_emin ();
    mpfr_set_emin (-1073);
    mpfr_init2 (y, 53);
    mpfr_set_d (y, x, MPFR_RNDN);
```

```

    int inex = mpfr_exp (y, y, MPFR_RNDZ);
    mpfr_subnormalize (y, inex, MPFR_RNDZ);
    double ret = mpfr_get_d (y, MPFR_RNDN);
    mpfr_clear (y);
    mpfr_set_emin (emin);
    return ret;
}

// htr.c's dd_exp: exp(x) at 161 bits, split into three doubles
void
eval_mpfr (double *h, double *l, double *s, double x)
{
    mpfr_t t;
    mpfr_init2 (t, 3*53+2);
    mpfr_set_d (t, x, MPFR_RNDN);
    mpfr_exp (t, t, MPFR_RNDN);
    *h = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *h, MPFR_RNDN);
    *l = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *l, MPFR_RNDN);
    *s = mpfr_get_d (t, MPFR_RNDN);
    mpfr_clear (t);
}

```

C Annex: Program 3

dbl.h is shown with Program 2 (Section 6).

```

// The parameters of eval_fix.c.

#include <stdint.h>

#define FIX_P 200 // bits after the point of every fixed-point number
#define FIX_TAB 256 // table size: x = (256 k + j) ln2/256 + r
#define FIX_LG 32 // extra bits of ln2/256, so that N * ln2/256 is exact
// to 2^-FIX_P for |N| < 2^32
#define FIX_DEG 13 // degree of the Taylor polynomial of exp(r)
#define FIX_NL 8 // a number is FIX_NL limbs of 32 bits, lowest first:
// 256 bits, in two's complement when it has a sign

// Program 3, the proved evaluator: exp with integers only, self-contained:
// no library, not even libm; doubles are read and built from their bits
// (dbl.h).
// Tang's scheme:
// x = N ln2/256 + r, N = 256 k + j, 0 <= j < 256, |r| <= 2^-9.5
// exp(x) = 2^k * 2^(j/256) * exp(r)
// A fixed-point number is an integer X standing for X / 2^FIX_P.
// Designed error (not yet proved): 2^-169 on y, 2^-158 after the split.

#include "dbl.h"
#include "fix.h"
#include "eval.h"
#include "fix_table.h"

// -----

```

```
// The integers: FIX_NL limbs of 32 bits, lowest first. Every operation is
// modulo  $2^{(32 \text{ FIX\_NL})}$ , so a signed number is in two's complement. The
// sizes in fix_exp stay below  $2^{213}$ , so nothing wraps.
```

```
typedef uint32_t num[FIX_NL];
```

```
static void
num_set (num r, const num a)
{
    for (int k = 0; k < FIX_NL; k++) r[k] = a[k];
}
```

```
// r = v, for  $0 \leq v < 2^{64}$ 
static void
num_set_u64 (num r, uint64_t v)
{
    for (int k = 0; k < FIX_NL; k++) r[k] = 0;
    r[0] = (uint32_t) v;
    r[1] = (uint32_t) (v >> 32);
}
```

```
static void
num_add (num r, const num a, const num b)
{
    uint64_t c = 0;
    for (int k = 0; k < FIX_NL; k++) {
        c += (uint64_t) a[k] + b[k];
        r[k] = (uint32_t) c;
        c >>= 32;
    }
}
```

```
static void
num_sub (num r, const num a, const num b)
{
    uint64_t c = 1; // a - b = a + ~b + 1
    for (int k = 0; k < FIX_NL; k++) {
        c += (uint64_t) a[k] + (uint32_t) ~b[k];
        r[k] = (uint32_t) c;
        c >>= 32;
    }
}
```

```
static void
num_neg (num r, const num a)
{
    num z = { 0 };
    num_sub (r, z, a);
}
```

```
static int
num_isneg (const num a)
{
    return a[FIX_NL - 1] >> 31;
}
```

```

// the sign of a - b, both read as nonnegative
static int
num_cmp (const num a, const num b)
{
    for (int k = FIX_NL - 1; k >= 0; k--)
        if (a[k] != b[k]) return a[k] < b[k] ? -1 : 1;
    return 0;
}

// the number of bits of a >= 0; 0 for a = 0
static int
num_bitlen (const num a)
{
    for (int k = FIX_NL - 1; k >= 0; k--)
        if (a[k]) {
            int b = 32;
            while (!(a[k] >> (b - 1))) b--;
            return 32 * k + b;
        }
    return 0;
}

// r = a 2^s, for a >= 0
static void
num_shl (num r, const num a, int s)
{
    num t = { 0 };
    int q = s / 32, b = s % 32;
    for (int k = FIX_NL - 1; k >= q; k--) {
        uint64_t v = (uint64_t) a[k - q] << b;
        if (k - q - 1 >= 0 && b) v |= a[k - q - 1] >> (32 - b);
        t[k] = (uint32_t) v;
    }
    num_set (r, t);
}

// r = floor (p / 2^s) for p >= 0 of n limbs, whose quotient fits in
// FIX_NL limbs; returns 1 when the bits dropped are not all 0
static int
limbs_shr (num r, const uint32_t *p, int n, int s)
{
    int q = s / 32, b = s % 32, nz = 0;
    for (int k = 0; k < q && k < n; k++) nz |= p[k] != 0;
    if (b && q < n) nz |= (p[q] & ((1u << b) - 1)) != 0;
    for (int k = 0; k < FIX_NL; k++) {
        uint64_t v = k + q < n ? p[k + q] >> b : 0;
        if (b && k + q + 1 < n) v |= (uint64_t) p[k + q + 1] << (32 - b);
        r[k] = (uint32_t) v;
    }
    // the quotient fits: nothing above bit 32 FIX_NL + s
    if (FIX_NL + q < n) assert ((p[FIX_NL + q] >> b) == 0);
    for (int k = FIX_NL + q + 1; k < n; k++) assert (p[k] == 0);
    return nz;
}

// r = floor (a b / 2^s), for a, b >= 0; returns 1 when the bits dropped

```

```

// are not all 0
static int
num_mulshr (num r, const num a, const num b, int s)
{
    uint32_t p[2 * FIX_NL] = { 0 };
    for (int i = 0; i < FIX_NL; i++) {
        uint64_t c = 0;
        for (int j = 0; j < FIX_NL; j++) {
            c += (uint64_t) a[i] * b[j] + p[i + j];
            p[i + j] = (uint32_t) c;
            c >>= 32;
        }
        p[i + FIX_NL] = (uint32_t) c;
    }
    return limbs_shr (r, p, 2 * FIX_NL, s);
}

// r = floor (a b / 2^s), for a >= 0 and b of any sign
static void
num_mulshr_s (num r, const num a, const num b, int s)
{
    if (!num_isneg (b)) {
        num_mulshr (r, a, b, s);
        return;
    }
    num mb, one;
    num_neg (mb, b);
    int nz = num_mulshr (r, a, mb, s);          // floor (-v) = -(ceil v)
    num_set_u64 (one, nz);
    num_add (r, r, one);
    num_neg (r, r);
}

// r = floor (a / d), for a >= 0 and 0 < d < 2^32
static void
num_divu (num r, const num a, uint32_t d)
{
    uint64_t rem = 0;
    for (int k = FIX_NL - 1; k >= 0; k--) {
        uint64_t v = (rem << 32) | a[k];
        r[k] = (uint32_t) (v / d);
        rem = v % d;
    }
}

// the len <= 64 bits of a >= 0 from bit pos up
static uint64_t
num_bits (const num a, int pos, int len)
{
    num t;
    limbs_shr (t, a, FIX_NL, pos);
    uint64_t v = t[0] | (uint64_t) t[1] << 32;
    return len == 64 ? v : v & ((1ul << len) - 1);
}

// -----

```

```

// Tang's scheme on these integers.

static num C[FIX_DEG + 1]; // 1/i!, rounded down
static num inv; // 256/ln2, with FIX_LG bits after the point
static num rmax; // bound on |r|: 181/256 2^-9 < 2^-9.5
static num err; // bound on the error of y: 2^-160

void
fix_init (void)
{
    num_set_u64 (C[0], 1);
    num_shl (C[0], C[0], FIX_P);
    for (int i = 1; i <= FIX_DEG; i++) // floor (floor (a/b) / c)
        num_divu (C[i], C[i - 1], i); // = floor (a / (b c))
    num_set_u64 (rmax, 181);
    num_shl (rmax, rmax, FIX_P - 17);
    num_set_u64 (err, 1);
    num_shl (err, err, FIX_P - 160);
    num_set (inv, fix_inv);
}

// exp(x) = y / 2^FIX_P * 2^k; returns k
static long
fix_exp (num y, double x)
{
    num X, r, t, u;
    // |x| = m 2^(be - 1075) from the bits of x; X = |x| 2^FIX_P, exactly
    union dbits b;
    b.d = x;
    int neg = b.u >> 63, be = (b.u >> 52) & 0x7fff;
    uint64_t m = b.u & ((1ul << 52) - 1);
    assert (be != 0 || m == 0); // x is 0 or normal
    if (be) m |= 1ul << 52;
    int sh = be - 1075 + FIX_P;
    assert (be == 0 || sh >= 0);
    num_set_u64 (X, m);
    if (be) num_shl (X, X, sh);
    // N = floor (|x| 256/ln2 + 1/2) = (floor (2 |x| 256/ln2) + 1) / 2
    num_mulshr (t, X, inv, FIX_P + FIX_LG - 1);
    long N = (long) ((num_bits (t, 0, 64) + 1) >> 1);
    if (neg) { N = -N; num_neg (X, X); }
    // r = x - N ln2/256, checked small
    num_set_u64 (u, N < 0 ? -N : N);
    int nz = num_mulshr (t, fix_ln2, u, FIX_LG); // floor (|N| ln2/256)
    if (N < 0) { // floor (-v) = -(ceil v)
        num_set_u64 (u, nz);
        num_add (t, t, u);
        num_neg (t, t);
    }
    num_sub (r, X, t);
    if (num_isneg (r)) num_neg (t, r); else num_set (t, r);
    assert (num_cmp (t, rmax) < 0);
    // exp(r) by Horner's rule, one truncation per product
    num_set (y, C[FIX_DEG]);
    for (int i = FIX_DEG - 1; i >= 0; i--) {
        num_mulshr_s (y, y, r, FIX_P);
    }
}

```

```

    num_add (y, y, C[i]);
}
assert (!num_isneg (y));
// times 2^(j/256)
long j = ((N % FIX_TAB) + FIX_TAB) % FIX_TAB;
num_mulshr (y, y, fix_T[j], FIX_P);
// y is not within err of a power of 2, so its exponent is that of exp(x)
num_sub (t, y, err);
num_add (u, y, err);
assert (num_bitlen (t) == num_bitlen (u));
return (N - j) / FIX_TAB;
}

// the top 53 bits of y, as a double scaled by 2^sc; y keeps the rest
static double
top53 (num y, long sc)
{
    int sh = num_bitlen (y) - 53;
    if (sh < 0) sh = 0;
    uint64_t hi = num_bits (y, sh, 53);
    num t;
    num_set_u64 (t, hi);
    num_shl (t, t, sh);
    num_sub (y, y, t);
    return dscale ((double) hi, sh + sc);
}

void
eval_fix (double *h, double *l, double *s, double x)
{
    num y;
    long k = fix_exp (y, x);
    *h = top53 (y, k - FIX_P);
    *l = top53 (y, k - FIX_P);
    *s = top53 (y, k - FIX_P);
}

// exp(x) from below: y - err, cut down to 53 bits. The assert checks
// that y + err stays below the next 53-bit value, so d + ulp(d) > exp(x).
double
low_fix (double x)
{
    num y, t;
    long k = fix_exp (y, x);
    num_sub (y, y, err);
    int sh = num_bitlen (y) - 53;
    uint64_t hi = num_bits (y, sh, 53);
    double d = dscale ((double) hi, sh + k - FIX_P);
    num_set_u64 (t, hi + 1);
    num_shl (t, t, sh);
    num_add (y, y, err);
    num_add (y, y, err);
    assert (num_cmp (y, t) < 0);
    return d;
}

```

The constants come from `gen_fix.c` (with MPFR); they are not trusted, since each is certified. This version gives bit for bit the same h , l , s and d as an earlier one written with GMP, on 10^6 random points from -634 to 709.7 (**measured**).

D Annex: the test

`make test` compares, line by line, the candidates and the hard cases of `htr.c`, Program 2 with MPFR, and Program 2 with Program 3 (`htr3`, linked with nothing but `libc`, for `main.c`). **Measured**, 2026-09-28:

slice, 10 000 chunks	candidates	result
from 0.25 up	410 325	identical; one hard case; none undecided
from -0.25 down	498 078	identical; one hard case; none undecided

Timing, on the first slice (**measured**, medians of five alternated runs, `cpu0`, turbo off):

program	time
<code>htr.c</code>	7.21 s
Program 2 + MPFR	7.97 s
Program 2 + Tang	7.23 s

Program 2 + MPFR is 10% slower because its `eval_check` is `dd_exp`: each candidate is evaluated at 161 bits, where `htr.c`'s `check` uses $53 + m = 88$ (reasoned from the code, not profiled). Program 2 with Tang costs no more than `htr.c`.

There are many candidates because the drift, about 2^{48} units, is much wider than the target 2^{29} (**computed**): about 41 per chunk, as in `htr.c`'s comment (**read**).

E Annex: limits

- `get_uint64` needs the slope below 1: $|x| < 1/2$ (**computed**). Fix: keep only its fractional part.
- $[x_0, x_1)$ must sit in one binade of x and of $\exp x$.
- Below $x \approx -708$, $\exp x$ is subnormal; above 709.78 , it overflows.
- Three doubles hold 158 bits only for $x \geq -634$ (**computed**); at $x = -700.3$ they keep 66 (**measured**). Below, Program 2 must take y itself.
- `eval` is called once per chunk, 0.7 ms of inner loop (**computed**), so its speed does not matter; the whole range is about 12 core-years (**computed**).